A JAVA-BASED ADAPTIVE MEDIA STREAMING ON-DEMAND PLATFORM

Giancarlo Fortino, Angelo Furfaro, Wilma Russo
DEIS, Università della Calabria
Via P. Bucci s/n, 87036 Rende (CS), Italy
E-mail: {g.fortino, w.russo, a.furfaro}@unical.it

Juan C. Guerri, Ana Pajares, Carlos E. Palau
Universidad Politecnica de Valencia
Camino de Vera s/n, 46022 Valencia, Spain
E-mail: {cpalau, jcguerri, apajares}@dcom.upv.es

KEYWORDS
VoD, Java Media Framework, RTP, RTSP, IP-multicast.

ABSTRACT

Nowadays, media streaming architectures and systems are gaining focus due to the widespread diffusion of IP-based, bandwidth-capable digital networks which can really support multimedia data-intensive on-demand services. In this paper, we present a Media on-Demand system which provides adaptive, multicast, and collaborative media streaming. Media streaming relies on the Real time Transport Protocol whereas streaming control is centered on the Real Time Streaming Protocol. A distinguishing feature of the proposed system is the ability of the media streamer component at the server side to adapt the media flow on the basis of the RTCP feedback of the client. The paper also describes the Java-based implementation of the system which uses the Java Media Framework library.

INTRODUCTION

Multimedia, in particular audio and video, is already being distributed in the Internet and in many other communications networks and systems (Steinmetz and Nahrstedt 1995). Thanks to many different products like RealNetwork's RealSystem (Realsystem 2001), Nullsoft's SHOUTcast (ShoutCast 2001), Windows Media (WinMedia 2001) or Apple's Quicktime (Quicktime 2001), the idea of broadcast has been brought to the web. Besides there are other multimedia applications based on the requests of the customers which are generally known as Video on-Demand systems (VoD). A VoD system allows customers to select movies from a large menu in order to view them (e.g., at home) at times of their choosing. Any VoD system is formed by three key structural blocks: the server, the communications network and the client. The main goal of a VoD system is to serve thousand of clients simultaneously. Each client connects to the communications network and gets in contact with the server to request a movie at a specific time (Wright 2001, Zink et al. 2001).
Today's information servers not only provide simple text-based data, but also a rich combination of text, still-images, animations, audio and videos. The main issues a VoD system has to deal with are storage spaces, bandwidth, synchronization, transport protocols, playback control protocols and scalability. There are several VoD systems developed using different techniques, architectures and philosophies, and each of them solve the above problems by means of alternative approaches.
There are several applications of VoD systems like broadband TV broadcast, education, medicine (Brett 1996, Guerri et al. 2000, Huang and Hu 2000). All these applications are usually based on the control of several multicast multimedia flows using a control protocol like RTSP (Shulzrinne et al. 1998).
Interesting example of VoD-like systems over Internet are: the KOM-player (Zink et al. 2001), Video Conference Recording On-demand (ViCRO) (Fortino and Nigro 2000), JMFVoD (Belda et al. 2002), multicast Multimedia-On-Demand (mMOD) (Parnes et al. 1997), and Universal Video-On-Demand (UVoD) (Lee 1999).
The main goal of this paper is to present the architecture and a prototype implementation of our Java-based media on-demand system providing multicast and adaptive streaming. Media on-Demand (MoD) is richer than Video on-Demand (VoD) in that MoD (Horn et al. 2001) provides for multimedia sessions which not only consist of synchronized audio and video streams, but also of other types of media such as text-tips, text-tickers, images, animations, slides, etc. To sum up, the playback can be a multimedia composite session.
The MoD system proposed centers on the following key technologies:

(i) *RTP as media transport protocol*. RTP (Schulzrinne et al. 1996), the Real-time Transport Protocol framework provides end-to-end delivery services for data with real-time characteristics. These services are suitable for various distributed applications that transmit real-time data, such as interactive audio and video. The companion protocol (RTCP) provides feedback to the RTP sources in the RTP session and to all the participants in the session. The same underlying transport protocol (i.e., UDP) is used for both, but different ports are used to differentiate packet streams;

(ii) *RTSP as playback control protocol*. The playback of the multimedia information is controlled by means of the Real Time Streaming Protocol (Schulzrinne et al. 1998). RTSP is an application-layer protocol that provides control over the delivery of real-time data. The protocol is typically applied for control over continuous time-synchronized streams of media. It usually acts as a remote control protocol for media servers. Our system makes the typical use of RTSP, not to deliver media by itself but to control streams delivered by RTP;

(iii) *Java Media Framework (JMF)*. JMF (Sun Jmf 2002) is an API defined jointly by Sun MicroSystems, Inc. and IBM Corporation. The main aim of the product is the provision of real-time delivery and control of multimedia data (video and audio) by means of streaming techniques. The API can be used in applets and in stand-alone applications developed in JAVA. A part from providing facilities for data processing (codecs, multiplexers and demultiplexers, renderers, etc) JMF allows transmission and reception of multimedia data using RTP in unicast and multicast modes, and the utilization of several capture devices like webcams. There are different versions of the API, the latest one is v. 2.1.1a, although new improvements are being included;

(iv) *Adaptive Server-to-Client streaming*. Adaptive streaming (Busse et al. 1996) is based on a media streaming rate control mechanism relying on RTCP feedback. The adaptation is performed in terms of bandwidth, losses and data encoding;

(v) *Friendly Applet-based client interface*. The choice to implement the client as a Java Applet allows easy system extendibility and dynamic software distribution.

The remainder of the paper is structured as follows. The second section introduces the main components of the media streaming on-demand architecture. The third section describes the RTSP

protocol and the media control component based on it. The adaptive streaming component is introduced in the forth section. In the fifth section an extension of the architecture to support tightly coupled groups of distributed clients is discussed. Finally conclusions and directions of further and on-going work are reported.

THE BASIC ARCHITECTURE OF THE MEDIA STREAMING ON-DEMAND SYSTEM

As has been commented in the introduction section, there are several IP-based streaming systems developed in the literature (Gebhard and Lindner 2001, Huang and Hu 2000, Rowe 2001). At the same time new multimedia libraries and APIs are appearing or are being improved, like Java Media Framework (Sun Jmf 2002) in order to be used to develop such systems.
A main aim of our work is the validation of the viability of JMF libraries for the implementation of a media streaming on-demand system. The execution of the system will allow the real evaluation of the management mechanisms for multimedia sessions, RTP/RTCP protocols, the compression standards included in the libraries and the utilization of RTSP. The developed prototype will serve as basis for the future evaluation of new multimedia protocols that will substitute the traditional simulation studies in this field.

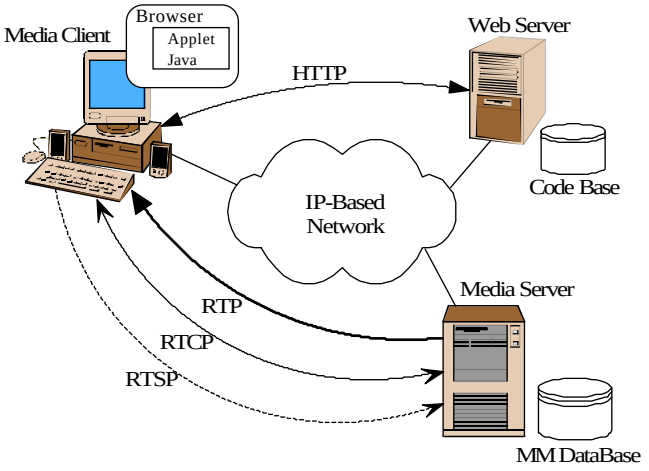

Figure 1: Architecture of the MoD system

With reference to a MoD system, four elements can basically be differentiated (Figure 1): client, web server, multimedia database and media server.

- *Client*: its development is based on a Java applet that is started each time the correspondent HTML page is loaded from the web server. Through the graphical user interface, the users can select from the remote multimedia database the multimedia session title they want to playback.
- *Web server*: it stores the start page and the Java applets. It also holds the authentication procedures and the database access management.
- *Database*: it keeps the information catalogue of the available multimedia sessions (e.g., movies).
- *Media Server*: it stores the media files and send them to the clients as soon as they request them.

Software structure

The Media on Demand (MoD) streaming system (Belda et al. 2002, Fortino and Nigro 2000) consists of three super blocks or applications (Figure 2) that communicate with each other through TCP and UDP –based connections, apart from the multimedia database containing the multimedia files published in the system.

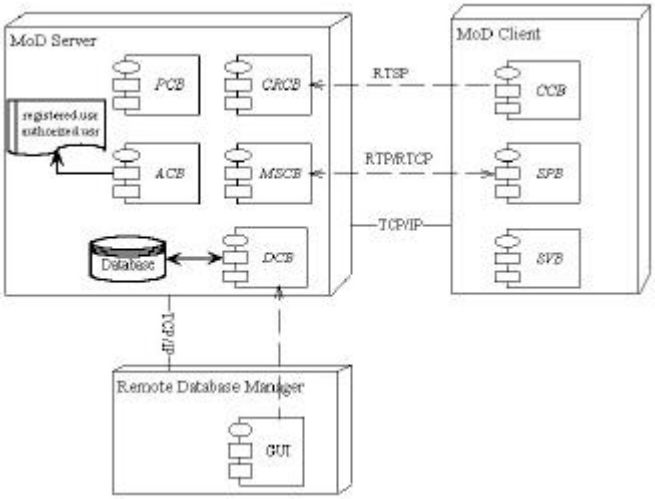

Figure 2: Components and Communications structure

- The *MoD streaming server*, continuously listens at a TCP port waiting for a client to make a connection request. The server exchanges messages defined according to the RTSP protocol with the clients through the opened sockets.
- The *MoD client*. There are two kinds of entities that can make a connection request: the client itself and the remote database manager. Both clients are applets and run in web navigators.
- The *Remote Database Manager*. The Database Manager is SQL based and provides access depending on the identity of the users. Some easy commands have been developed in order to communicate the server application with the database.

The server allows a fixed number of clients to simultaneously access to the system. It also includes a user authentication procedure along with an application that allows the list of users that can access the system to be managed.

Database structure

The published files database used by the application has been structured in three tables:

- *Main table*: contains the published multimedia files names and other related attributes.
- *Categories table*: contains the names of the categories to which a multimedia file can belong to.
- *Subcategories table*: contains the names of the subcategories to which a multimedia file can belong to.

Control protocol

The system control protocol consists of the exchanged messages and rules between the streaming MoD server and both the client and the database remote manager. The control protocol is based on RTSP and is described in the next section.

Server application

It is a multi-threaded unit whose main blocks are:

- The Connection Requests Control Block (CRCB) is created when the server is activated from the administrator's GUI. When it receives a connection request it creates a front-end handling the connection with the client.
- The Multimedia Streams Control Block (MSCB) provides the mechanisms to process multimedia files for creating and controlling the RTP sessions needed to send the multimedia streams to the client that has requested them.
- The Parameters Control Block (PCB) provides the mechanisms to read and write the configurable parameters of the MoD streaming system.
- The Database Control Block (DCB) accesses to, reads and modifies the database, by means of SQL instructions, using the JDBC-ODBC bridge driver provided by the library java.sql.
- The Access Control Block (ACB) manages two files: "registered.usr" and "authorized.usr". Both of them contain login names and passwords of the users along with their access rights.

Client Application

The client application is structured in the following blocks:

- The Control Connection Block (CCB) is committed for sending and receiving control messages exchanged between the client and the server.
- The Streams Player Block (SPB), it is responsible for streams synchronization and reproduction.
- The Statistics Viewer Block (SVB) gets the RTP sessions statistics from the SPB.

Remote database manager

The remote database manager is an applet embedded in a html page. When the applet is loaded, it generates a graphical user interface identical to the one generated by the local database manager. The database changes introduced by the user are sent through a TCP control connection to the server. This connection is established on the applet initialization. If there is not another user managing the database, the server will send the database content to the remote manager side through the TCP control connection block.

STREAMING CONTROL

RTSP: an overview

The Real-time Streaming Protocol (RTSP) (Schulzrinne et al. 1998) is a text-based application level protocol developed for on-demand delivery control of media streams both live and pre-recorded. It establishes and controls either a single or several time synchronized streams of continuous media such as audio or video. In other words, RTSP acts as a "network remote control" for multimedia servers.
RTSP extends HTTP 1.1 by introducing a stateful behavior and new methods. Thus, RTSP requires the notion of session. It defines a session identifier which is chosen by the server at the session establishment and used throughout the session lifetime. RTSP can be based on a single TCP connection activation, on multiple activations (each one for a request/reply interaction), on UDP.

Multimedia presentations are identified by URLs, using a protocol scheme of "rtsp". The hostname is the server containing the presentation; whilst the port (the default port is the 554) indicates which port the RTSP control requests should be sent to. Presentations may consist of one or more separate streams. The presentation URL provides a means of identifying and controlling the whole presentation rather than coordinating the control of each individual stream. So, the *rtsp://asimov.deis.unical.it:4044/Movie01/videotrack* URL, identifies the video stream within the presentation Movie01, which can be controlled on its own. If the user would rather stop and start the whole presentation, including the video, then he/she would use the *rtsp:// asimov.deis.unical.it:4044/Movie01/* URL.
The standard RTSP methods are:

- The DESCRIBE method is sent from the client to the server and allows retrieving the description of a presentation or a single media object.
- The SETUP method issued by the client causes the server to generate the session identifier and allocate resources for the session. During the setup phase, client and server can negotiate transport parameters (addresses and ports) as well as media streaming parameters.

- The PLAY method signals the server to start streaming. PLAY can have a range header which contain a time parameter: start and stop instants within the multimedia session. Time can be encoded in NPT (Normal Play Time), SMPTE (relative time), and UTC (absolute time).

- The PAUSE method interrupts the media streaming delivery temporarily.

- The TEARDOWN method stops the streaming so that the server can free the resources allocated to the session.

- The SET_PARAMETER method allows setting a parameter associated to the current session.

- The GET_PARAMETER method retrieves a value of a parameter of a session.

- The OPTIONS method returns the supported methods of a server.

- The REDIRECT method informs a client that it must connect to the new server location contained in the Location header.

- The RECORD method allows starting the recording of an announced multimedia session.

- The ANNOUNCE method posts on the server the description of a multimedia presentation (e.g., to be recorded).

In addition, new methods, which extend the RTSP functionality, can be purposely introduced as reported in (Fortino and Nigro 2000, Fortino et al. 2001).
Figure 3 reports a typical control session of a video playback which consists of request/reply pairs according to the RTSP format.

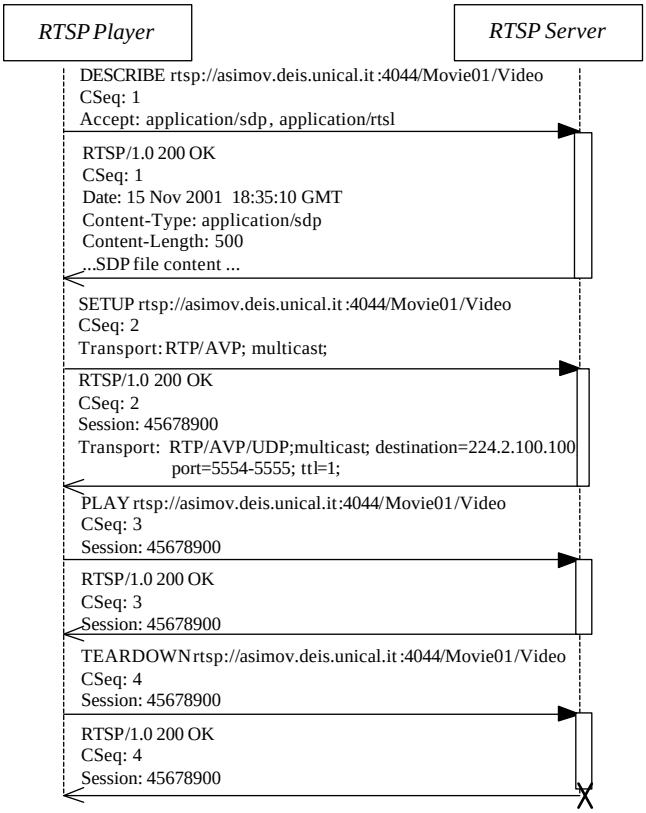

Figure 3: The UML sequence diagram of the RTSP-based control session of a multicast video stream

RTSP inside JMF

RTSP support has recently been added to the Java Media Framework library. This integration enables JMF based clients (both applets and applications) to communicate with RTSP enabled servers and request the streaming of specified mediafile. For instance, the Java Multimedia Studio application can be used as an RTSP client by opening an RTSP URL from the File menu. RTSP within JMF library adds more control mechanisms to multimedia information playback control.

JMF 2.1.1a allows to build RTP/RTSP players which interoperate with standard-based, third-party video streaming servers such as Sun StorEdge Media Central Streaming Server (Sun Storedge 2001) and Apple Quicktime Streaming Server (Apple Quicktime 2001).

Figure 4 depicts a simplified UML class diagram of the RTSP classes and interfaces which are inside JMF. The *RtspUtil* class can be employed by a programmer to implement an RTSP Player or an RTSP Server. It implements the *RtspListener* interface which defines two methods. In particular, the *rtspMessageIndication* method is invoked by the *RtpManager* when a new RTSP message arrives. The *RtpManager* class creates and manages the socket connections, and makes it available the *sendMessage* method to transmit a message over a connection identified by a specific ID. It's worth noting that the *RtspUtil* class is associated to one or more *RtpManagers*. If the *RtspUtil* is used as Player, the *RtpManager* instances are created after the SDP description of the requested media file has been retrieved and the SETUP message is being issued. In order to realize an RTSP Server we have first modified and then extended the *RtspUtil* class respectively by introducing a per-connection server state variable and by specializing the *processRtspRequest* method.

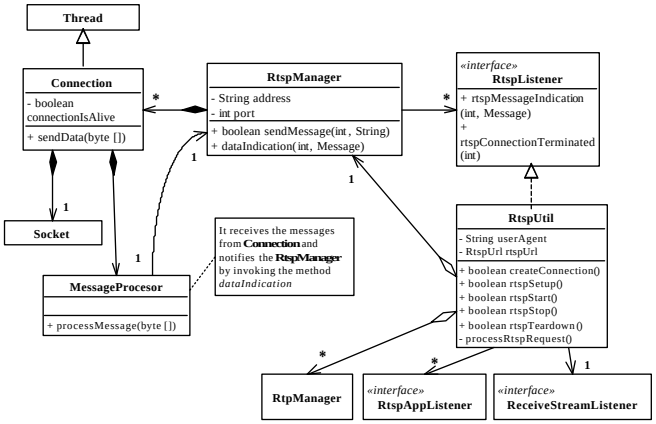

Figure 4: Class Diagram of the JMF implementation of RTSP

STREAMING ADAPTATION

A media streaming system (Dutta and Schulzrinne 2001) over Internet has to provide flexibility in coping with the bandwidth variations so as to deliver a certain degree of quality of service. In fact, in "traditional" design of integrated services networks, multimedia applications such as videoconferencing tools can negotiate a desired QoS during the connection setup phase and then the network guarantees such a quality. Conversely, Internet doesn't provide bandwidth guarantees, thus application level mechanisms are to be employed. In particular, we exploit a network-initiated in-call QoS adaptation control which bases the application target data rate on network feedback (Busse et al. 1996). Starting from a given bandwidth, low losses lead the application to slowly increase the media streaming bandwidth, while high packet losses lead to bandwidth decrease. The control algorithm is mainly based on the feedback information conveyed by the receiver reports of the RTCP companion protocol. This feedback information allows the source of the media streaming based on RTP to estimate the loss rates experienced by the receiver/s and to adjust its bandwidth accordingly.

RTCP: Receiver Reports and SDES reports

After receiving several data packets, the RTP-based client calculates some performance measurements (or statistics) which are then sent back to the media streaming component at the server site in the Receiver Report (RR) through the RTCP channel. The statistics are the following:

(i) *Total Packet Lost (TPL)*: the total number of packet lost, since the transmission beginning. It is computed from the number of packet actually received and the number of packets expected, which is detected on the basis of the sequence number of the last RTP data packet arrived.

(ii) *Fraction of Packet Loss (FPL)*: the fraction of the packet lost during the time interval between two consecutive transmissions of RR reports.

(iii) *Interarrival Jitter (IJ)*: the mean deviation (smoothed absolute value) of the difference in packet spacing at the receiver compared to the sender for a pair of packets (Schulzrinne et al. 1996).

Besides, RTCP messages contain an SDES (Source DEScription) packet which embeds information to identify the data sources such as the canonical name (CName), email address, telephone number, application specific info and alert messages.

End-to-End application control mechanism

The adaptive streaming control scheme (Busse et al. 1996, El-Marakby and Hutchinson 1997) is based on the following three phases:

(i) *RTCP analysis*. The RR of the receiver/s (in the case of unicast or multicast streaming) are analyzed.

(ii) *Network state estimation*. The actual network congestion state seen by the receiver/s can be classified as unloaded, loaded of congested (or simply as congested or uncongested).

(iii) *Bandwidth adjustment.* The bandwidth of the media stream is regulated according to the classification of the network state analysis.

Our adaptive streaming controller relies on the FPL statistics which is the indicator of the congestion. FPL can be used either raw or filtered. In the latter case, by applying a low-pass filter it is possible to smooth the FPL so reducing QoS oscillations. In the following two algorithms are described. The first is based on a basic algorithm proposed in (El-Marakby and Hutchinson 1997). The second is a variant of a more sophisticated algorithm proposed in (Busse et al. 1996). In figure 5 the basic algorithm Java-like is portrayed.

```
FPL=lastRR.getFPL();
if (FPL>lambda)
  currRate=Math.max(alpha*currRate, minRate);
else
  currRate=Math.min(beta*currRate, reqRate);
```

Figure 5: Basic rate control algorithm

```
FPL=lastRR.getFPL();
fpl_LAST=(1-a)*fpl_OLD+a*FPL;
if (fplLast>lambdaC)
  currRate=Math.max(mu*currRate, minRate);
else
if (fplLast<lambdaU)
  currRate=Math.min(currRate+chi, reqRate);
```

Figure 6: Rate control algorithm with filter and deadzone regulator

Lambda is the threshold parameter whose exceeding (keeping below) can cause a decrement (increment) of the media streaming rate. Conversely, [minRate, reqRate] is the admissible rate variation range. minRate is chosen by the streamer component according to the involved media whereas reqRate is the rate chosen by the client at media streaming setup time. Alpha is the fixed decrement fraction coefficient whereas beta is the fixed increment fraction coefficient. It is worth noting that: (i) no RTCP analysis is performed; (ii) the network state estimation (congested or not) is driven by lambda; (iii) bandwidth adjustment is done by using the max/min featured functions. In figure 6 the second algorithm Java-like is reported.

The algorithm first uses a smoothing (low-pass) filter, then a linear regulator with dead zone which classifies a receiver in congested, loaded and unloaded.

The introduced algorithms strongly depend on the choice of their characterizing controller parameters (*a*, *mu*, *chi*, *lambdaC*, *lambdaU*) in order to be really effective.

In case of unicast media streaming the presented algorithms can be directly applied. In case of point-to-multipoint media streaming, the decision to increase, decrease or hold the rate should be taken by considering all the receivers. A simple solution is to adapt the media flow to the slowest receiver so as to avoid congestion for all the receivers. The main drawback is that faster receivers obtains poor quality. A more interesting solution is to compute the percentages of the congested, unloaded and loaded receivers and to make a decision of rate decreasing, holding or increasing on the basis of these percentages (e.g, if the % of congested receivers is greater than 10, then the rate is decreased).

Another solution can be to adjust the media streaming by considering only the network conditions of the media playback owner, i.e., who requested the playback.

More powerful approaches to the streaming adaptation are the video gateways and the layered coding schemes supported by lightweight and tunable feedback protocols such as SCUBA (Scalable ConsensUs-based Bandwidth Allocation) (Amir et al. 1997).

GROUP SHARING OF A REMOTE MEDIA SESSION CONTROL

In traditional VoD systems, remote control of the media session can be only applied by the session owner. In case the requested media streaming is multicast, other clients can tune in and watch the multimedia session. However, they don't have session control and their view is affected by the control commands issued by the session owner client. In order to allow for sharing the session control among a group of clients, a shared media session controller is to be realized. In (Fortino et al. 2001) several schemes (unicast, hybrid and multicast) to implement such a virtual controller are proposed.

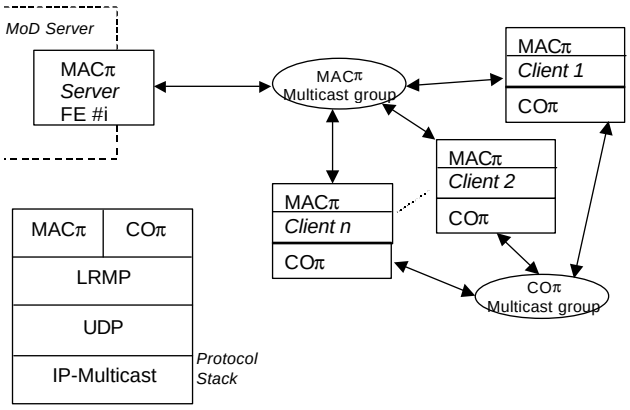

Figure 7: The VCC architecture and the protocol stack

Key issues concern with the media streaming control protocol and behavioral policy. The former has to provide rules to enable each client within a group to consistently send control commands. The latter is needed to regulate participants which exhibit behavior mining the media presentation view such as frequent pauses or seeks.

A first prototype of the virtual control component (VCC) relies on the MACπ protocol which is a multicast extension of the RTSP protocol and the COπ protocol supporting a voting-based coordination policy (Fortino et al. 2001).

Figure 7 portrays the distributed architecture of the VCC. For each requested media session to be shared, a Front-End (FE) at the MoD server site is spawn on a given multicast group. Each media client activates a COπ component on another multicast group. After the shared session is started, i.e., a media client issued the PLAY request and the FE replied to all, each command before being sent is constrained by the coordination policy adopted. The current available policy is based on a voting mechanism which is triggered by the seek (i.e., PLAY with range header) and the TERMINATE method (the PAUSE is not constrained). A seek is accepted if and only if the majority of the

media clients agree whereas the TERMINATE needs the unanimity to be raised.

CONCLUSIONS AND FUTURE WORK

In this paper, we have described the basic architecture of a Java-based platform providing media on-demand services. The platform mainly relies on: (i) the multimedia internetworking protocols, namely RTP, RTSP, SAP and SDP; (ii) Java and the Java Media Framework.

Interesting feature of our platform is that it enables adaptive streaming based on receiver-based feedback algorithms and allows for sharing playback among multiple clients. The choice of Java as implementation language is strategic not only because it provides rich and powerful object-oriented libraries for distributed and multimedia computing but also since it allows to easily deploy software systems over heterogeneous environments like the Internet. The proposed MoD platform is the first prototype which will be used as base for the development of a cooperative and content-enriched media on-demand system over the Internet for distance learning and entertainment.

Our on-going and future work aims at: (i) refining the prototype and evaluating its performance; (ii) defining a comprehensive protocol suite for collaborative media on-demand which is mainly oriented to the learning and entertainment application domains.

ACKNOLEDGEMENTS

We would like to thank our colleagues at Università della Calabria and Universidad Politécnica de Valencia, particularly A. Belda, M. Esteve, L. Nigro, and F. Pupo for numerous helpful discussions and ideas. This research is supported in part by the Spanish and Italian Governments in the frame of a bilateral project.

REFERENCES

Amir E.; S. McCanne; and R. Katz. 1997. "Receiver-driven Bandwidth Adaptation for Light-weight Sessions." In *Proceedings of ACM Multimedia*, Seattle, WA, (Nov).

Apple Quicktime. 2001. Apple Quicktime Streaming Server, available at http://www.apple.com.

Belda A.; A. Pajares; J. C. Guerri; C.E. Palau; and M. Esteve. 2002. "JMFVoD: Design and Implementation in Java of a Media Streaming on Demand System." In *Proceedings of SCS Euromedia*, Modena, Italy, (Apr).

Brett P. 1996. "Using Multimedia: an Investigation of Learners' Attitudes." *Computer Assisted Language Learning* 1 No. 2-3, 191-212.

Busse I.; B. Deffner; and H. Schulzrinne. 1996. "Dynamic QoS Control of Multimedia Applications based on RTP." *Computer Communications* 19, No. 1, (Jan).

Dutta A. and H. Schulzrinne. 2001. "A Streaming Architecture for Next Generation Internet." In *Proceedings of ACM Multimedia.*

El-Marakby R. and D. Hutchinson. 1997. "Delivery of Real-time Continuous Media over the Internet." In Proceedings of the 2nd IEEE Symposium on Computers and Communications (ISCC'97), Alexandria, Egypt, (Jul).

Fortino G. and L. Nigro. 2000. "ViCRO: an interactive and cooperative videorecording on demand system over Internet Mbone." *International Journal Informatica* 24, No. 1, 97-105.

Fortino G.; L. Nigro; and F. Pupo. 2001. "An MBone-based on-demand system for cooperative off-line learning." *Proceedings of IEEE Euromicro*, Warsaw, Poland, (Sep).

Gebhard H. and L. Lindner. 2001. "Virtual Internet Broadcasting." *IEEE Communications* 39, No 6 (Jun), 182-188.

Guerri J. C.; M. Esteve; C. Palau; M. Monfort; and M.A. Sartí, "A software tool to acquire, synchronise and playback multimedia data: an application to kinesiology." *Computer Methods and Programs in Biomedicine* 62 (Jan), 51-58,.

G.B. Horn, P. Knudsgaard, S.B. Lassen, M. Luby and J.E. Rasmussen. 2001. "A Scalable and Reliable Paradigm for Media on Demand." *IEEE Computer Magazine* 34, No. 9 (Sep), 40-45.

Huang S. and H. Hu. 2000. "Integrating Windows streaming media technologies into a virtual classroom environment." In *Proceedings of the International Symposium on Multimedia Software Engineering*, 411–418.

Lee J. 1999. "UVoD: An Unified Architecture for Video-on-Demand Services." *IEEE Communications Letters* 3 No.9 (Sep), 277-279.

Parnes P. et alter. 1997. "multicast Multimedia On-Demand (mMOD)." Available at http://mmod.cdt.luth.se.

Realsystem. 2001. RealNetworks's RealSystem, available at http://www.real.com

Rowe L. A. 2001. "Streaming Media Middleware is more than Streaming Media." In *Proceedings of ACM Multimedia.*

Schulzrinne H.; S. Casner; R. Frederick; and V. Jacobson. 1996. "RTP: A Transport Protocol for Real-Time Applications." *IETF RFC-1889*, (Jan).

Schulzrinne H.; A. Rao; and R. Lanphier. 1998. "Real Time Streaming Protocol (RTSP)." *IETF RFC 2326*, (Apr).

ShoutCast. 2001. NullSoft's ShoutCast, available at http://www.shoutcast.com.

Steinmetz R. and K. Nahrstedt. 1995. "Multimedia: Computing, Communications and Applications." *Prentice Hall.*

Sun Jmf. 2002. Java Media Framework, available at http://java.sun.com/products/java-media/jmf/index.html

Sun StorEdge. 2001. Sun StorEdge Media Central Streaming Server, available at http://www.sun.com.

WinMedia. 2001. Microsoft Windows Media, available at http://www.microsoft.com.

Wright W. E. 2001. "An efficient video on-demand model." *IEEE Computer*, (May), 64-70.

Zink M.; C. Griwodz; and R. Steinmetz. 2001. "KOM Player – A Platform for Experimental VoD Research." In *Proceedings of ISCC'01*, Hammamet (Tunisia).